

Documentación Proyecto Subida Nota

Introducción de la Aplicación

Mi app de subida de nota es una aplicación móvil diseñada para innovar en la forma en que los usuarios gestionan su día a día. A través de la gamificación, mi app transforma las rutinas aburridas, las tareas pendientes y la actividad física en una experiencia similar a la de un videojuego. El usuario asume el papel de, gracias a completar tareas rutinarias, poder subir de nivel, ganar XP y mantener rachas diarias completando objetivos.

El **propósito** principal es:

- Combatir la procrastinación y el sedentarismo mediante un sistema de recompensas psicológicas.
- Fomentar la consistencia en los hábitos diarios mediante un sistema de rachas,
- Incentivar la actividad física básica integrando un podómetro nativo, que es lo único que no he puesto en funcionamiento por falta de tiempo (en la app sale el diseño del podómetro pero con mock data).
- Proporcionar un gestor de tareas que sea visualmente estimulante e intuitivo.
- Ofrecer una sensación de progresión real a través de niveles, rangos y recompensas.

Contexto y Justificación

En la actualidad, mantener la concentración y la motivación es un desafío constante debido a la sobreestimulación digital. Las aplicaciones de productividad tradicionales suelen ser planas, aburridas y, a menudo, se abandonan a las pocas semanas. Por otro lado, las aplicaciones de salud suelen centrarse únicamente en datos fríos y estadísticos. Mi app nace de la necesidad de fusionar la productividad y la salud en un entorno inmersivo, utilizando la gamificación como el núcleo central de la experiencia del usuario.

Competencia

Si bien existen en el mercado aplicaciones de productividad gamificadas (como Habitica) y cientos de contadores de pasos, mi app destaca por una propuesta de valor única: no existen aplicaciones en el mercado que combinen una interfaz de diseño "neón/cyberpunk" al estilo de los sistemas de los RPG modernos, con un gestor de tareas avanzado y un

podómetro integrado. Mientras otras apps utilizan diseños caricaturescos, minimalismo extremo, simuladores de RPG con interfaces simulando píxeles... mi app apuesta por una estética oscura, paneles de neón azul y púrpura, y una sensación de sistema.

Flujo de la app

La navegación de mi app está dividida en cuatro pantallas principales (Fragments), accesibles a través de un menú de navegación inferior. El flujo está diseñado para ser rápido y permitir acciones inmediatas:

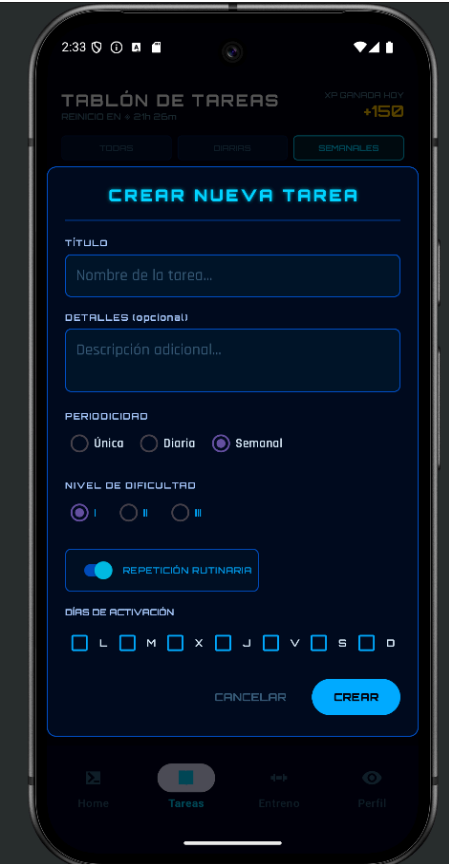
A. Pantalla Principal (Home) Es el panel de control del usuario. Aquí se muestra un resumen de la información más crítica: el nivel actual, título, los días de racha, un resumen del progreso semanal de los hábitos, las tareas más urgentes por hacer y un registro del sistema (log) que notifica los últimos eventos importantes (como ganar XP o subir de nivel).

B. Gestor de Tareas (Tareas) En esta sección, el usuario puede ver la lista completa de sus misiones. Permite crear, editar, filtrar (diarias, semanales, únicas) y completar las tareas. Cada tarea muestra su dificultad, recompensas y fecha de caducidad.

C. Actividad Física (Entreno) Este apartado está dedicado a la salud física del jugador. Funciona como un podómetro que cuenta los pasos diarios, mostrando el progreso en una barra circular de estilo neón, recompensando al usuario al alcanzar metas de movimiento.

D. Perfil del Jugador (Perfil) Aquí reside el componente RPG más puro. El usuario puede ver sus estadísticas totales (XP, monedas, tareas completadas en la historia), visualizar los logros desbloqueados y gestionar su inventario o equipamiento adquirido.

(CAPTURAS DE PANTALLA EN SIGUIENTE PAGINA)



Explicación de Tecnologías Utilizadas

- **Android SDK (Java):** El proyecto está desarrollado de forma nativa para Android, garantizando un rendimiento óptimo y acceso directo a los sensores del dispositivo (a pesar de que el podómetro no he llegado a ponerlo en funcionamiento).
- **Room Database:** Se utiliza la librería Room de Android Jetpack para la persistencia de datos. Room actúa como una capa de abstracción sobre SQLite, permitiendo gestionar la base de datos local del dispositivo mediante anotaciones y objetos, lo que hace que el manejo de la información sea mucho más seguro y estructurado, a continuación se detallará cómo ha sido utilizada.

Detalles Técnicos de Implementación

A continuación, se detallan algunas de las decisiones arquitectónicas y técnicas más relevantes tomadas por simplicidad y rendimiento durante el ciclo de vida del desarrollo:

Arquitectura de Base de Datos (Room sin Repositories)

Para la gestión de la base de datos local, se ha optado por utilizar Room. Room facilita enormemente la creación de bases de datos relacionales transformando las tablas en Entidades (Clases). En este proyecto, por razones de simplicidad y agilidad en el desarrollo, se ha decidido no utilizar el patrón Repository. En su lugar, se invoca a los DAOs (Data Access Objects) directamente desde los Fragments o Activities. En cada entidad (como o Usuario.java) se definen los campos exactos de la base de datos, y en los DAOs se estructuran directamente las queries SQL necesarias (inserciones, actualizaciones, o conteos específicos).

Uso de Hilos y Concurrencia (Executors)

Room prohíbe por defecto ejecutar consultas a la base de datos en el hilo principal (Main Thread) para no bloquear la interfaz de usuario (UI) y provocar errores ANR (Application Not Responding). Para solucionar esto sin recurrir a librerías externas complejas, se ha hecho un uso extensivo de la clase Executors.

Por ejemplo, al crear o completar una tarea, se utiliza un hilo secundario temporal:

```
Executors.newSingleThreadExecutor().execute(() -> {
    // 1. Operación en segundo plano (No bloquea la pantalla)
    database.tareaDao().insertar(nuevaTarea);

    // 2. Volver al hilo principal para actualizar la interfaz visual
    requireActivity().runOnUiThread(() -> {
        cargarTareasPendientes();
        Toast.makeText(context, "Tarea creada",
            Toast.LENGTH_SHORT).show();
    });
});
```

Esta estructura garantiza que la aplicación se mantenga fluida; la base de datos guarda la información "en la sombra", y solo cuando termina, avisa a la pantalla para que recargue las listas y muestre las notificaciones correspondientes.

Manejo de Fechas y Periodicidad (Clase Calendar)

El sistema de tareas de mi app permite definir misiones Únicas, Diarias o Semanales. Para calcular dinámicamente las fechas límite y de regeneración sin depender de librerías de terceros, se hace un uso intensivo de la clase nativa Calendar de Java. Al crear una tarea, el sistema lee la periodicidad elegida y manipula la fecha:

- Si es Diaria, el Calendar ajusta la hora límite a las 23:59:59 del día actual.
- Si es Semanal, el algoritmo calcula cuántos días faltan para el próximo domingo e inyecta esa suma al calendario, estableciendo esa fecha como límite.

Diseño de Interfaz: Múltiples Drawables y Assets

Gran parte del tiempo de desarrollo se invirtió en la creación de la identidad visual del sistema. Para lograr el estilo "Sistema/RPG Neón", no se utilizaron imágenes pre-renderizadas, sino que se programaron múltiples archivos Drawable XML. Esto incluye:

- Fondos de contenedores y paneles brillantes, como @drawable/bg_neon_panel o @drawable/bg_card_panel_azul.
- Diseños para los estados de botones y pestañas (activos/inactivos).
- ProgressBars personalizadas: Se han diseñado barras de progreso complejas para representar la barra de Experiencia (XP) y el porcentaje circular del podómetro, utilizando capas (layer-list) con gradientes luminosos que reaccionan de manera nativa al progreso numérico, logrando una estética altamente inmersiva y con un coste de rendimiento mínimo.

Desarrollo, Problemas y Mejoras a Futuro

Desarrollo y Problemas: El mayor reto durante el desarrollo fue lograr una sincronización perfecta entre la interfaz de usuario (UI) y la base de datos sin congelar la aplicación. Además, el diseño de la interfaz consumió gran parte del tiempo, ya que fue necesario crear la estética neón desde cero mediante archivos XML para evitar el uso de imágenes pesadas que ralentizaran la app. El manejo de la regeneración de tareas rutinarias (diarias y semanales) requirió una lógica compleja de comprobación de fechas para evitar duplicados.

Mejoras a Futuro:

- Implementar copias de seguridad en la nube (Cloud Sync) mediante Firebase para guardar base de datos en nube y así no perder tus datos de usuario.
- Añadir un sistema de "Equipamiento" con objetos que se compran en la tienda mediante monedas (las monedas se ganan completando tareas según dificultad) y estos puedan ser guardados en el perfil junto a tu personaje.
- Incorporar notificaciones push locales que avisen antes de perder una racha o cuando una tarea esté a punto de expirar.

Programa en Python

Por último, he desarrollado un programa externo de Python que permite analizar los datos exportados directamente desde la base de datos de la aplicación, y generar un informe en formato Excel (utiliza tanto Pandas como Openpyxl). Este informe incluye estadísticas relevantes como nivel del usuario, progreso al siguiente nivel, distribución de tareas por dificultad, progreso diario de XP y pasos, así como un historial de registros del sistema. La herramienta, diseñada con una interfaz sencilla utilizando Tkinter, se conecta directamente al archivo SQLite exportado de la app, procesando los datos reales del usuario para ofrecer análisis personalizados y proyecciones a futuro, todo con un formato claro y visualmente organizado.

Para sacar la BD: Android Studio → Device Explorer →
data/data/com.example.app_subida/databases/app_subida_database → botón derecho →
Save As.